The YIN Pitch Estimation Algorithm

j0ey-code
[REDACTED]

[REDACTED]
25 April 2026

Abstract

The purpose of this paper is to serve as an introductory exploration and discussion of YIN, a powerful, yet elegant and simple time-domain based algorithm utilized for monophonic pitch detection. YIN improves significantly upon previous methods of frequency estimation (predominantly, autocorrelation) while still maintaining the same foundational approach: determining the frequency by comparing and / or contrasting with a cross-section of delayed (or "lagged") versions of the signal.

Introduction

Every sound produced by a musical instrument or the human voice is composed of multiple frequencies vibrating simultaneously, but among them is one that defines the perceived note: the fundamental frequency (F_0), commonly referred to as *pitch*. In Western music, all melody and harmony are built from just 12 tones (A, A#/Bb, B, C, C#/Db, D, D#/Eb, E, F, F#/Gb, G, G#/Ab), each corresponding to a specific fundamental frequency – a measurable, quantifiable sound wave that persists even as additional harmonics and overtones ring out around it. The same principle applies to speech and singing: the pitch of a speaker or vocalist's voice is determined by the fundamental frequency at which their vocal cords vibrate, surrounded by the harmonic content that gives each voice its unique timbre. Accurately detecting this fundamental frequency within a

complex signal is, therefore, a problem that spans both music and speech processing, and one that has proven more difficult than it might seem.

In 2002, Alain de Cheveigné and Hideki Kawahara officially published their work on the YIN algorithm: "YIN, a fundamental frequency estimator for speech and music" **[1]**. Named after the Chinese concept of duality (yin and yang, together known as the "taijitu" – opposites flowing into one another) the algorithm's name itself is, in part, an allusion to the process which it performs. While earlier pitch detection algorithms focused mostly on *autocorrelation, the YIN algorithm balances out some of the errors inherent to autocorrelation by using cancellation to help ignore harmonic interference and amplitude spikes.

The YIN algorithm is specifically designed with *monophonic pitch detection in mind, but this does not mean the process is straightforward or simple. Standard autocorrelation methods had failed, for a long time, to consistently and accurately detect the fundamental frequency of a note being played, even on instruments which were monophonic in nature / by design (i.e. bass guitar, trumpet, vocals, saxophone, etc.). Thus, a better approach was necessary.

*autocorrelation: method to detect pitch by measuring and comparing a waveform / signal to delayed versions of itself

*monophonic: refers to one / a single note sounding at a time, as opposed to multiple notes in sequence, simultaneously (i.e. polyphonic)

Background Information: Autocorrelation Fundamentals

Before we can really explore the YIN algorithm, it is important to first understand standard autocorrelation as an approach to pitch detection. Autocorrelation, while certainly presenting a few key strengths and advantages, also has some significant drawbacks as well. Several problems are apparent in standard autocorrelation, including, but not limited to...

- **octave errors:** inaccurate octave readings by doubling or halving the true period.
- **formant filtering:** when tracking the *formants in human speech, the first formant (F_1) can tend to produce a greater peak than the fundamental frequency (F_0), leading to cascading errors in the pitch detection of human vocals.
- **window size selection:** an appropriate window (analysis frame) must be selected to perform autocorrelation – too small, and the pitch can't be picked up; too large, and more continuous audio signals (like human speech) will vary too much and ruin the accuracy.
- **low frequency inaccuracy:** standard autocorrelation methods can also fail to detect frequencies which are too low (or, "bass-y") *because* of window size selection that is too small (lower frequencies have longer periods) - this is most apparent in instruments such as the tuba, double / upright bass, or the bass trombone.

*formant: resonant frequencies of the vocal tract (particularly in humans) that contribute to the overall sound of speech

These persistent and troublesome issues with autocorrelation methods have been known for some time; in February of 1977, electrical engineer Lawrence R. Rabiner described and outlined all these faults in his paper "On the Use of Autocorrelation Analysis for Pitch Detection" within the IEEE Transactions on Acoustics, Speech, and Signal Processing, vol. 25, no. 1. Rabiner noted that while autocorrelation generally produces a prominent peak at the *pitch period, formant structure in the signal often creates competing peaks – and the choice of analysis window only compounds the problem, since the windowing process tapers the autocorrelation function toward zero at higher indices, giving those lower-index formant peaks even greater relative magnitude (Rabiner, pg. 1) **[2]**. This can result in false positive errors, where the wrong peak is selected as the estimated pitch period. Rabiner also provides the true, mathematical definition for the autocorrelation of a signal...

$$\phi_x(m) = \lim_{N\to\infty} \frac{1}{2N+1} \sum_{n=-N}^{N} x(n)x(n+m).$$

Figure 1.) Rabiner's general formulation for autocorrelation of a signal; m is incremented to shift the signal, while n then walks through the signal to accumulate the shifted signal's sum – the result is a sequence of numbers, each representing one value per signal lag

This is the *theoretical* definition, as one may note from the lim(N → ∞); under ideal conditions, we would just continually measure the waveform. However, as stated previously, an analysis window **must** be selected, and this is not obvious anywhere in Rabiner's definition. In fact, quite the opposite, as beneath the sigma / summation notation (Σ), $n = -N$ signifies integer values for the signal as existing from anywhere between -∞ and +∞. Before the sigma / summation notation is the necessary normalization for such a formula; after all, we aren't looking

*pitch period: the duration of time for a single periodicity of the fundamental frequency to complete; reciprocal of the frequency

for a sum – we are searching for the mean approximation of the fundamental frequency. As such, the limit is taken, and division by the number of terms in the summation is performed in order to find the average similarity in frequency per sample.

Let's also take a quick look at the more practical, applied, time-domain based implementation of Rabiner's formulation, presented by the creators of the YIN algorithm themselves, Cheveigne and Kawahara.

$$r'_t(\tau) = \sum_{j=t+1}^{t+W-\tau} x_j x_{j+\tau}$$

Figure 2.) Cheveigne and Kawahara's derived, practical implementation of the autocorrelation formula **[1]**

Let's step through each part of this equation / summation, as to better understand how autocorrelation works in a pragmatic sense, before finally moving onto the YIN algorithm itself.

$r'_t(\tau)$ represents the autocorrelation value at a given lag τ, starting from position t in the signal. The index j walks from $t + 1$ up to $t + W - \tau$, where W is the window size. The "$- \tau$" in the upper bound pulls it back just enough to ensure the summation never exceeds the end of the frame. At each step, the signal value at position j is multiplied by the signal value at the shifted position $j + \tau$. A positive product indicates agreement between the original and shifted signals, while a negative product indicates disagreement. Summing these products across the entire window yields a single number representing the overall similarity at that particular lag. This process is then repeated for $\tau = 1, 2, 3$, etc., producing an autocorrelation curve; the lag value where the curve peaks corresponds to the estimated period, and the inverse at that point gives the estimated fundamental frequency.

Yang to Yin: Improving on Autocorrelation

For decades people struggled to perfect autocorrelation pitch detection and estimation methods, only considering how by comparison of waveform signals (the "yang" methodology) we can estimate the fundamental frequency of a sound signal. Besides the other issues with autocorrelation, these approaches have yet another pitfall in that audio signals are not naturally constant in amplitude – they swell, crescendo, decay, and fade or cut out. This was the single biggest problem in autocorrelation, because a larger amplitude spike within a single frame of audio may falsely be detected as the frequency to estimate, resulting in the chosen lag / delay / shift value to be too large, and therefore the resulting frequency will be too low. Conversely, if an audio signal decays / fades / rings out within its given frame, the algorithm picks a shorter lag value which yields frequencies that are too high. Autocorrelation asked "how similar is the signal from a shifted version of itself?", which only compounds the errors caused by variations in peaks and amplitudes. YIN's essential change is the implementation of a difference function rather than a similarity function – YIN instead asks the question, "how different is the signal from a shifted version of itself?".

This plain difference function constitutes only step two of the YIN algorithm (step 1, for Cheveigne and Kawahara's purposes, being standard autocorrelation). In figure 3 is a table directly from their paper **[1]** which shows the significant error mitigation using this step alone as opposed to simply autocorrelation (step 1), which had a 10% observed gross error rate (90% success rate).

TABLE I. Gross error rates for the simple unbiased autocorrelation method (step 1), and for the cumulated steps described in the text. These rates were measured over a subset of the database used in Sec. III. Integration window size was 25 ms, window shift was one sample, search range was 40 to 800 Hz, and threshold (step 4) was 0.1.

Version	Gross error (%)
Step 1	10.0
Step 2	1.95
Step 3	1.69
Step 4	0.78
Step 5	0.77
Step 6	0.50

Figure 3.) Table I from Cheveigne and Kawahara's paper **[1]** showing the diminishing error rate in accurate pitch estimation at each corresponding step of the YIN algorithm

And below, we can see this step two (the core difference function) shown in plain mathematics.

$$d_t(\tau) = \sum_{j=1}^{W} (x_j - x_{j+\tau})^2,$$

Figure 4.) Cheveigne and Kawahara's defined difference function for an unidentified period; the summation of differences squared

While autocorrelation computes the inner product of the signal with a shifted copy of itself (a similarity metric that *peaks* at the correct period), the difference function instead computes the sum of squared differences – a dissimilarity metric that reaches its *minimum* at the correct period. This fundamental change in the process, from peak-finding to valley-finding, is YIN's greatest improvement. In autocorrelation, the correct period must compete with amplitude spikes, formant peaks, and harmonic interference that can all inflate the similarity score at incorrect lags, creating false positives that are difficult to distinguish from the actual peak. This is avoided completely when utilizing a difference function, since the point closest to zero is a value that only the true period can approach. A correctly shifted copy of the signal will consistently

cancel out the original signal, sample for sample. At the correct lag point, every subtraction will yield something near zero because the signal and its delayed copy are *nearly* identical. At an *incorrect* lag point however, these subtractions end up producing larger and larger values that the squaring and summing will only amplify, thereby ruling them out entirely.

Yet there is still an issue. This core difference function is still reflective of a range of values accumulated over the entire summation and, because of its very nature, comes with an inherent bug. $\tau = 0$ produces a false minimum; that is, the lag / tau value at zero for each iteration will yield a mathematical tautology (i.e. meaningless output). Of course if the lag is zero there will be **zero** real difference measured. A band-aid solution might be to simply start τ at 1 each time, and this would still admittedly result in a gross-error rate of only 1.95% in practice / implementation. This is unfortunately only a minor symptom of the real bug plaguing our core difference function. The actual issue is that we have no normalization!

$$d'_t(\tau) = \begin{cases} 1, & \text{if } \tau = 0, \\ d_t(\tau) \Big/ \left[(1/\tau) \sum_{j=1}^{\tau} d_t(j) \right] & \text{otherwise.} \end{cases}$$

Figure 5.) Cumulative mean normalized difference formula / function [CMNDF]; post-processing phase to divide $d_t(\tau)$ by the current running average [$(1/\tau)$ x Σ] of d_t computed values up until that point

The cumulative mean normalized difference function (CMNDF, step three of YIN) provides an even more refined approximation framework for honing in on the fundamental frequency, within the complex mess of "noise" that encompasses an audio signal. The core difference function feeds us critical information about how different the waveform is at such and such a lag; the CMNDF uses this raw data to extrapolate further by re-framing it again, this time instead as "how different is this audio signal at τ lag, relative to how different it typically is?".

By normalizing the crude signal information with the CMNDF, YIN can then further refine the audio to come to an even more precise estimation. Steps four (absolute thresholding), five (parabolic interpolation), and six (best local estimate) of the YIN algorithm resolve any remaining bugs or real-time implementation specific issues (octave errors, discrete vs. real sampling, additional possibilities for false positives, etc.). The first of these post-processing steps, the thresholding, fixes a certain number as a decision gate to test the received data from the CMNDF against. It iterates through the elements, much like many of the functions mentioned insofar, and tests whether each value falls below the thresholding gate.

For the *very first* signal value to fall below the threshold, we capture it and stop. In the implementation described by Cheveigne and Kawahara, this absolute value is set to 0.1, "With a threshold of 0.1, the error rate drops to 0.78% (from 1.69%) as a consequence of a reduction of ''too low'' errors accompanied by a very slight increase of ''too high'' errors." (Cheveigne & Kawahara, pg. 3) **[1]**. The absolute thresholding step is designed predominantly to correct octave errors – particular false positives caught by the algorithm because of their specific interval relationship to the fundamental frequency, i.e. a sub-harmonic. Parabolic interpolation (step five) is deployed to conduct even more audio refinement, using the neighboring delayed signal values. Since real-world problems do not meaningfully exist in discrete terms (i.e. a frequency can fall on a decimal point), it is imprecise to implement it in a discrete fashion. A lag increment that only moves along integers, while still functional, is ultimately inaccurate and introduces a small but notable rounding error. To solve this, a parabola is retroactively fitted between the minimum values for the current waveform, and the two beside it. This parabola's vertex point provides a split, sub-sample from which a more precise estimate can be taken (e.g. 1024.7 instead of 1025).

Step six serves a similar purpose to step five; a sanity check more or less, but at a different level of the algorithm. All steps thus far have been concerned with only the audio signals in the selected analysis frame / window. As a final refinement, the best local estimate function now checks the neighboring analysis frames and determines if a value even *lower* than the one within the current frame was detected. If this is the case, a larger valley than the one caught in the current frame would be found at *around* the exact same lag value. This would mean the analysis frame selected was not the optimal one, but an adjacent window managed to catch a cleaner fragment of the signal. So, we disregard the old analysis frame value(s) and focus on the new value(s) found for the period's estimation, within the new window!

Implementation

The mathematics and abstraction of defining such an algorithm can tend to obfuscate the core process, which may otherwise be fairly simple to understand. To bring things full circle, we will examine a couple of the critical operations at play in the YIN algorithm, and how they tie back into the foundations laid by Cheveigne and Kawahara in their paper. Below is pseudo-code for the most important part of YIN: the core difference function, in quadratic, O(n^2/4) time.

```
array* func computeDifference(signal, frameLen) {
	array differences[] = new array(frameLen / 2);
	// tau, lag delay sweep through frame
	for (tau = 0; tau < (frameLen / 2) - 1; tau++) {
		sum = 0;
		// j iterates to perform operation for each tau value
		for (j = 0; j < (frameLen / 2) - 1; j++) {
			delta = signal[j] - signal[j + tau];
			sum = sum + (delta * delta); // sum of differences squared!
		}
		differences[tau] = sum;
	}
	return differences;
}
```

Figure 6.) Pseudocode implementation for the sum of squared differences function – the "core difference" function, step two of the YIN algorithm

This accumulates our summations and saves them to some structure, `differences[]`, from which we can normalize the raw signal data using the CMNDF. If you'll have noted in figure 5 on page 8, we capture the outputs of our core difference formula / function to find mean normalized versions of our input signals. Here, our pseudocode maps one-to-one with this function by using the values captured within the `differences[]` structure as input for the next stage of the YIN algorithm's pipeline… the cumulative mean normalized difference function.

```
array* func CMNDF(differences, frameLen) {
	array avgDiffs[] = new array(frameLen / 2);
	avgDiffs[0] = 1; //
	accumulator = 0;

	for (tau = 1; tau < (frameLen / 2) - 1; tau++) {
		accumulator = accumulator + differences[tau];
		avgDiffs[tau] = differences[tau] / (accumulator / tau);
	}

	return avgDiffs;
}
```

Figure 7.) Pseudocode implementation for the cumulative mean normalized difference function, in linear, O(*n* / 2) time

Conclusion

It should be evidently apparent why the YIN algorithm is considered so revolutionary and ingenious. In many real-time applications, stability, speed, and efficiency are everything. But more importantly, so are accurate, precise, meaningful results! The core difference function is enough to have flipped the script on its own, but the application of the CMNDF and the additional post-processing phases are what make YIN truly stand out – reducing error rates from 1.95%, down to only 0.50%. For use cases within the realm of digital signal processing, where data can be hard to unscramble and analyze, YIN serves as a testament to the remarkable methods we can devise for solving complex, sometimes abstract, real-world problems.

Works Cited

[1] de Cheveigné, A., & Kawahara, H. (2002). "YIN, a fundamental frequency estimator for speech and music." The Journal of the Acoustical Society of America, 111(4), 1917–1930. https://doi.org/10.1121/1.1458024. Accessed 28 Mar. 2026.

[2] Rabiner, L. "On the Use of Autocorrelation Analysis for Pitch Detection." IEEE Transactions on Acoustics, Speech, and Signal Processing, vol. 25, no. 1, Feb. 1977, pp. 24–33, https://doi.org/10.1109/tassp.1977.1162905. Accessed 11 Apr. 2026.

[3] Rao, Prajwal S., et al. "A Comparative Study of Various Pitch Detection Algorithms." 2020 5th International Conference on Computing, Communication and Security (ICCCS), Oct. 2020, pp. 1–6, https://doi.org/10.1109/icccs49678.2020.9277448. Accessed 11 Apr. 2026.